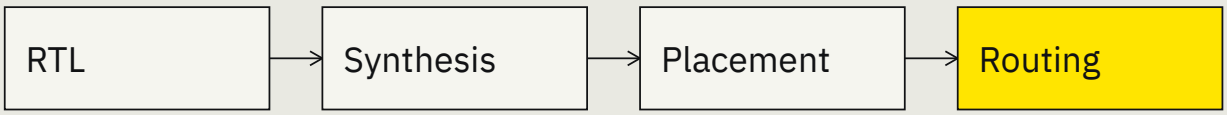
VOLUME 07 / 08

SYNTHESIS AND FPGA

Map the core, read timing and plan real board integration

Run open-source implementation, interpret resources and critical paths, and design a synchronous-memory board adaptation.

FIG 11 / IMPLEMENTATION FLOW



The final timing estimate belongs to a device, constraints, tools and route seed.

A hands-on companion to the RISC-V project. Examples, implementation steps, debugging and completion checks. Personal design edition / October 2026.

What you will build

You will map and route the measurement shell, interpret raw reports and understand what remains before a real FPGA demonstration.

Before you start

A passing functional regression from Volume 6. Know register-to-register timing.

Files and outputs

Makefile, rtl/timing_top.sv, scripts/metrics.py, build/synthesis.log, build/nextpnr.log and reports/implementation.json.

Build order

1. Choose the exact top and implementation boundary.
2. Run synthesis and route with recorded tools and constraints.
3. Read packed resources and the critical path.
4. Perform a controlled implementation comparison.
5. Plan clock/reset, synchronous memory and observable board results.

The checked-in core is an assistant-generated reference. Work on a learning branch. The code excerpts explain selected regions and are not complete replacement modules unless explicitly labeled.

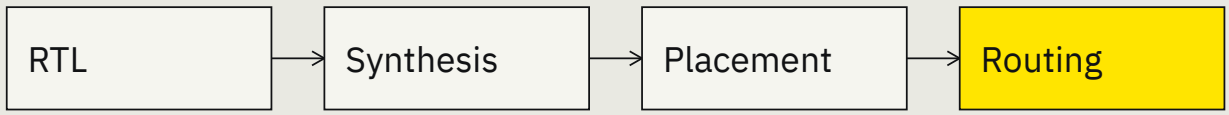
Synthesis maps logic, routing estimates timing

Synthesis turns RTL into a network of target logic resources. A lookup table, or LUT, implements a small Boolean function. Flip-flops store state. Dedicated RAM and DSP blocks can implement larger memories and arithmetic when the device provides them.

Place-and-route assigns resources to physical locations and connects them. Wire delay matters. A synthesis cell count alone cannot establish a routed clock frequency.

The measurement shell registers instruction/load inputs and folds core outputs into an observable register. It keeps the datapath alive during optimization. It contains no program memory, data memory or peripherals.

FIG 11 / IMPLEMENTATION FLOW



The final timing estimate belongs to a device, constraints, tools and route seed.

Measured reference: iCE40 HX8K CT256, requested 12 MHz. Over route seeds 1–5 the internal timing estimate has a median of 36.05 MHz (34.63–37.60). Mapping uses 4,012 LUT4 primitives and 1,692 flip-flops, packed into 5,539 logic cells.

Package I/O is unconstrained. The result is not external timing sign-off or a measured board clock. HX8K provides no hard DSP blocks, so DOT4 uses logic. FPGA resources are not ASIC area in square millimeters.

Clock frequency is only one part of performance

Cycles per instruction, or CPI, divides total cycles by retired instructions. A useful workload also has an instruction count. Execution time is approximately:

time = instruction count × CPI ÷ frequency

> **ILLUSTRATIVE EXAMPLE**
>
> A 100-instruction sequence needs 4 fill cycles, 10 load-use stalls and 16 cycles for eight taken redirects. Total: 130 cycles. CPI: 1.30. At an assumed 20 MHz, that is 6.5 microseconds.

DOT4 can reduce instruction count for packed dot products, but it can also add logic delay. Compare total cycles and each implementation's own clock estimate. Include packing and memory traffic when claiming end-to-end improvement.

RV32I has no integer multiply instruction. A fair scalar RV32I baseline needs a software multiply routine. Compiling a C multiply for RV32IM silently changes the comparison.

The reference benchmark does this. At N=256, DOT4 is 5.2× faster than a table-based RV32I multiply after its clock cost, and 22.2× faster than shift-and-add. Power was not measured.

Step 1: define what is being measured

`make synth` maps rv32_core alone. `make timing` maps timing_top, which adds registered external inputs and an observable output register. These are different tops with different resource costs.

The shell makes instruction/load inputs synchronous endpoints and keeps core outputs observable so optimization cannot discard the datapath. It does not execute a stored program.

Write the boundary explicitly: no instruction/data memories, UART, programmer, boot path or board constraints are included. A folded observation signal preserves hardware observability for implementation; it is not a correctness signature from a running application.

Choose HX8K CT256, the 12 MHz constraint and seed 1 to reproduce the reference. Record the exact tool versions and RTL hashes. Other devices or versions are separate experiments.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 7A. Why is comparing the LUT count from make synth to the count from make timing not a controlled DOT4-area experiment?

Step 2: run mapping and route

Use the supplied target rather than manually skipping its reporting stage.

```
docker run --rm --user "$(id -u):$(id -g)" \
  -v "$PWD:/work" rv32-tools make timing
```

Yosys reads both SystemVerilog hardware files and runs synth_ice40 for timing_top. nextpnr consumes the JSON netlist, chooses physical resources and routes the connections.

The command sets HX8K, package CT256, seed 1 and a 12 MHz constraint. Package I/O is allowed to remain unconstrained. The flow saves an ASC implementation file and a JSON timing report under build/.

Check the command exit status and the full log. A generated netlist alone is not a completed route. A route completing at a lower requested target does not establish timing at a previously quoted higher target.

The measurement parser archives the raw route report/log and a compressed synthesis log in reports/raw/.

Step 3: read resource counts correctly

Read synthesis primitive counts and packed place-and-route use as related but different views. A LUT, flip-flop and packed logic cell are not interchangeable units.

The reference shell maps to 4,012 LUT4 primitives and 1,692 flip-flops, packed into 5,539 of 7,680 HX8K logic cells. Its memories are external to the boundary, so zero block RAM use does not mean a complete system needs no memory.

HX8K lacks hard DSP blocks. The dot-product multiplication therefore uses logic. Resource use on a different family with DSPs can differ substantially.

Do not convert FPGA cell counts into square millimeters or energy without an appropriate physical model. Large resettable register arrays can also affect inference and mapping.

Open the JSON report and identify the keys for synthesis primitives, packed utilization, flip-flops, target and source hashes. Explain which count you would use in a README and why.

Step 4: trace the limiting timing path

Find the clock's achieved and constrained values in the route report. Then read the detailed slow path in the log: launch register, combinational cells/wires and destination register.

Classify the path before optimizing. It may involve the dot unit, a forwarding mux, address calculation, decode or register-file selection. Do not assume the largest-looking arithmetic block is the bottleneck.

Over route seeds 1–5 the reference's estimated internal Fmax has a median of 36.05 MHz (34.63–37.60) against a requested 12 MHz constraint. Each estimate belongs to one placement. The design was not board-tested at any of them.

Input/output timing is not signed off. The synchronous shell provides internal endpoints, but unconstrained package pins prevent a complete board-interface claim.

Propose one change that shortens the identified path, then rerun both functional regression and implementation. A timing improvement with incorrect execution is not a useful result.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 7B. Can a successful Yosys-only run justify “running at 50 MHz”?

Step 5: compare configurations fairly

Create an experiment branch or separate worktree. Compare a DOT4-enabled configuration against one that rejects DOT4, keeping top, memory boundary, device, tools, constraints and route seeds identical.

Record resource delta, clock estimates and the exact code change. Repeat a few route seeds to see placement variation. Keep each seed's raw report, including failures.

A feature-disabled core cannot run a DOT4 binary, so use an appropriate scalar program for workload comparison. Check the same mathematical outputs. Include scalar software multiplication under RV32I.

Convert cycles to estimated runtime using each configuration's own clock estimate. Report what is measured and what is inferred. The reference ablation finds DOT4 costs 19% more logic cells and 11% of median Fmax.

Keep negative results. If DOT4 saves cycles but lowers clock enough to erase the gain, that is an informative architecture result.

Step 6: adapt to synchronous FPGA memory

The reference expects combinational instruction/data reads. Typical FPGA block RAM returns data after a clocked read. Treat that as an interface change, not a wiring substitution. The reference board image sidesteps it by clocking block RAM on the falling edge, which halves the memory path budget (`docs/BOARD.md`).

For a first adaptation, choose either a fixed synchronous-read pipeline contract or a request/response protocol. Specify when a request is accepted and which saved instruction receives the response.

Preserve load address, size, destination and instruction identity until the response arrives. Extend the interlock so dependent work waits. Prevent repeated stores if a stage is held. Squash or discard responses associated with canceled younger requests.

Instruction fetch also needs response alignment: store the requested PC with the returned instruction, and discard a stale sequential response after a redirect.

Start by simulating a one-cycle memory wrapper. Then vary the response delay if your protocol supports it. Do not quote the old one-bubble rule or old timing numbers after this architectural change without rerunning the corresponding checks.

Step 7: make a board demonstration observable

Before buying or selecting a board, verify current availability, supported toolchain, FPGA capacity, package, oscillator, programmer and documentation. The existing HX8K shell uses much of the available logic; a smaller device may not fit.

A real top needs board pin assignments, a known clock, reset synchronization/treatment, memory/program loading and a visible result. A small ROM program can compute a signature and display pass/fail on an LED or send it over UART.

Simulate the complete board top and memory wrapper before programming. Check clock constraints and I/O timing for the actual board. Preserve the program, expected signature, configured clock and observed output.

Only after a checked program runs at the configured physical clock can you describe it as “running at X MHz.” A board photograph without program/output evidence is insufficient.

The reference has such a top: `make board` builds an HX8K Breakout image whose self-test passes in simulation, 38.21 MHz post-route. It has not run on a board.

A research experiment for this part

Write this entry before editing. It is a proposed experiment, not a result already obtained by the learner.

Question	Which path limits this configuration’s internal clock estimate?
Prediction	The detailed route report identifies a path whose logic or wiring can be shortened.
Experiment	Read the path, change one mechanism in a separate configuration, rerun correctness and the same implementation flow.
Evidence	Both revisions, identical target/constraints, seeds, resource reports and raw timing paths.

What to record

Save source revision, tool versions, input words, seed, exact command, expected and observed behavior, raw artifact path and limitation. End with the decision that the observation supports.

Use `docs/EXPERIMENT_LOG.md`. If the result contradicts the prediction, keep the failure and explain the next test. Do not rewrite the prediction afterward to make the experiment appear successful.

Answers and completion check

7A. They include different hardware boundaries. Keep the same shell and only vary the feature under investigation.

7B. No. Synthesis provides mapping evidence. Route supplies constrained timing estimates, and a physical board demonstration supplies observed operation at a configured clock.

Completion check

Reproduce the route, explain its limiting path and resource units, and state excluded hardware. For board work, finish the new memory contract and full-top simulation before making operating-frequency claims.

Sources and next steps

These sources support the concepts in this volume. The current RTL and local execution contract remain the authority for implementation-specific behavior.

1. Yosys documentation
2. nextpnr
3. Project source
4. RISC-V ISA

Detailed chapter-mapped reading is in `docs/textbook/SOURCES.md`. Research-interest context is in `docs/RESEARCH.md`. The original overview and its interactive figures remain companion material.

Keep the evidence boundary

Teaching labs are separate from the 253-program baseline. Reference coverage and timing numbers belong to the recorded source/tool configuration. No physical board execution, ISA certification or formal proof is implied. The DOT4 speedup covers one kernel.

Continue with Volume 8. Run the complete regression before treating a learning edit as a finished implementation.